

# 0.16. Deux rythmes, écrits en millisecondes

`ms(1000)` et `ms(250)` : chaque durée s'écrit en millisecondes, la console la traduit en images.

## 1. Ce qu'il faut comprendre

C'est le programme du 0.14, mais les durées ne sont plus des nombres d'images à calculer de tête : elles s'écrivent en **millisecondes** (millièmes de seconde). 1000 ms = 1 seconde ; 250 ms = un quart de seconde.

**Ce qui est nouveau ici** : `ms(durée)`. Elle fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer. Elle **traduit** une durée en millisecondes en nombre d'images, car la console ne sait compter que les images.

**La traduction** : une image dure  $1000 \div 60 \approx 16,7$  ms. `ms(1000)` donne donc 60 images, `ms(250)` donne 15 images, `ms(500)` donne 30 images. Le résultat est arrondi à l'image la plus proche : `ms(100)` donne 6 images, soit 100 ms environ.

**Elle ne coûte rien** : la traduction est faite par le compilateur, avant que la cartouche existe. `ms(250)` est remplacé par 15, et la console ne voit qu'un nombre. C'est pour cela qu'il faut écrire la durée **en clair** : `ms(250)`, et non `ms(une_variable)`.

**Les constantes** `DUREE_LENTE` et `DUREE_RAPIDE` rangent les deux durées en haut du programme : pour changer un rythme, on change un seul nombre, en millisecondes.

**La limite** : les compteurs sont des `uint8_t`, qui s'arrêtent à 255 images. On peut donc aller jusqu'à `ms(4250)`, environ 4 secondes. Au-delà, le compilateur refuse et l'explique. En dessous de 17 ms (moins d'une image), il refuse aussi : la console ne sait pas compter plus fin.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage — la mise en route de la console
main()         91 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`,

```
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser() × 2      24 o sur place, dans main()
    image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// _____
// LES DURÉES, EN MILLISECONDES (1000 ms = 1 seconde).
// ms() les traduit en images (60 images = 1000 ms) :
// ms(1000) = 60 images, ms(250) = 15 images.
// _____

#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <poser>       // pose une tuile sur une case du fond
```

## 2. UNE CONSTANTE — DUREE\_LENTE

```
const uint8_t DUREE_LENTE = ms(1000);    // la lente change toutes les 1000 ms (1 seconde)
```

## 3. UNE CONSTANTE — DUREE\_RAPIDE

```
const uint8_t DUREE_RAPIDE = ms(250);    // la rapide change toutes les 250 ms (4 fois par seconde)
```

## 4. UNE VARIABLE — LENTE

```
uint8_t lente = 0;                        // QUELLE lettre montre la lente : 0 = A ... 25 = Z
```

## 5. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;                      // QUELLE lettre montre la rapide : 0 = A ... 25 = Z
```

## 6. UNE VARIABLE — IMAGESLENTE

```
uint8_t imagesLente = 0;                // le CHRONOMÈTRE de la lente, en images
```

## 7. UNE VARIABLE — IMAGESRAPIDE

```
uint8_t imagesRapide = 0;               // le CHRONOMÈTRE de la rapide, en images
```

## 8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();                // attend l'image suivante : un tour ≈ 16,7 ms

        imagesLente++;          // une image de plus pour chaque chronomètre
        imagesRapide++;

        // — LA LENTE : quand son chronomètre atteint DUREE_LENTE (1000 ms) —
        if (imagesLente == DUREE_LENTE) {
            imagesLente = 0;      // son chronomètre repart de zéro
            lente++;              // la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // après Z, retour à A
        }

        // — LA RAPIDE : quand son chronomètre atteint DUREE_RAPIDE (250 ms) —
        if (imagesRapide == DUREE_RAPIDE) {
            imagesRapide = 0;     // son chronomètre repart de zéro
            rapide++;             // la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

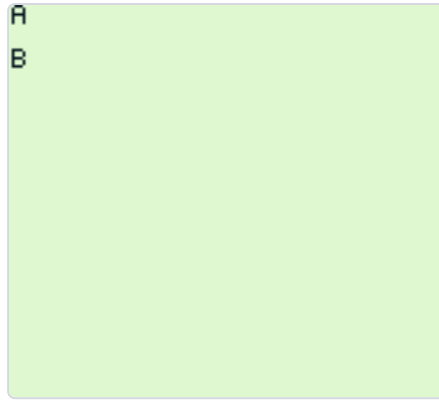
        poser(0, 0, ALPHABET[lente]); // ligne 0 : la lente (toutes les 1000 ms)
        poser(0, 2, ALPHABET[rapide]); // ligne 2 : la rapide (toutes les 250 ms)
    }
}
```

## 3. Ce qui se passe, étape par étape

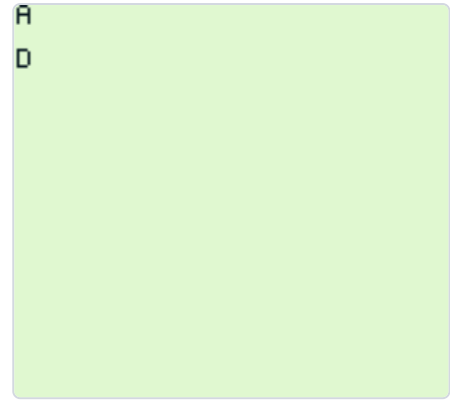
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



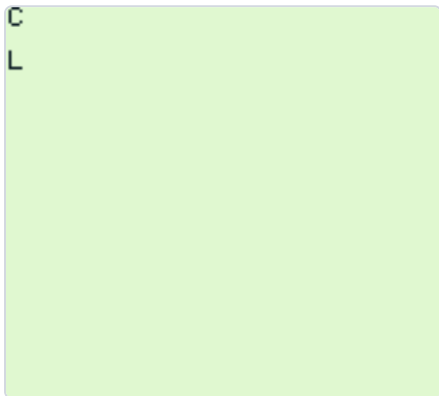
la toute première image — la console vient de démarrer — image 1



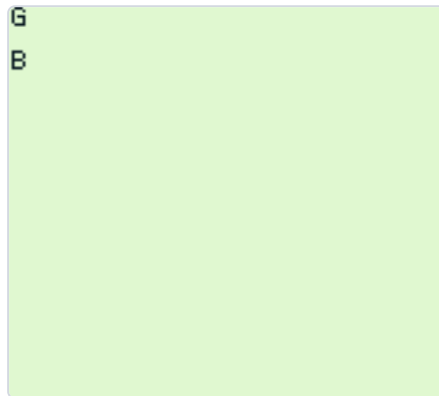
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

## 4. Ce qu'on doit voir

Comme au 0.14 : la lettre du bas change quatre fois pendant que celle du haut change une fois.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la rapide change toutes les 250 ms (8 fois en 2 s) (AB AC AD BE BF BG BH CI CJ)

✓ la lente change toutes les 1000 ms (2 fois en 2 s)

Ensuite : **0.16.1. Deux rythmes, écrits en millisecondes — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++